

Untitled

Assembly Transcriptions and Explanations

Program 1: Basic String Output

This program demonstrates the use of a label to store a string and the `PUTS` trap to print it.

Code snippet

```
.ORIG x3000          ; Program begins at memory address x3000 [cite: 96]
LEA R0, prompt1      ; Load the address of the label 'prompt1' into R0
[cite: 97]
PUTS                  ; Write the string at R0 address to the console [cite:
97, 373]
HALT                  ; Stop execution [cite: 98]
prompt1 .STRINGZ "Hello World!" ; Store "Hello World!" followed by a null
[cite: 99, 371]
.END                  ; End of assembly file [cite: 100]
```

Explanation The code starts by defining where it lives in memory using the `.ORIG` directive. It uses the `LEA` (Load Effective Address) instruction to find exactly where the text "Hello World!" is stored in the computer's memory and places that location into register `R0`. The `PUTS` command then looks at `R0` and prints every character it finds until it hits a zero (the null terminator). Finally, the `HALT` command tells the computer to stop running instructions.

Program 2: Adding Two Numbers

This program loads values from memory, adds them, and stores the result back into a designated memory location.

Code snippet

```
.ORIG x3000          ; Program starts at memory address x3000
MAIN LD R0, NUM1      ; Load value from NUM1 into R0
      LD R1, NUM2      ; Load value from NUM2 into R1
      ADD R2, R0, R1    ; Add R0 and R1, store result in R2
      ST R2, SUM        ; Store result from R2 into memory location SUM
```

```

        HALT                ; Stop execution

; Data Section
NUM1 .FILL #25              ; Define NUM1 with decimal value 25
NUM2 .FILL #17              ; Define NUM2 with decimal value 17
SUM  .BLKW #1               ; Reserve one memory word for the result
.END                        ; End of assembly file

```

Explanation This program is designed to perform basic math on data stored in memory. First, it uses the `LD` instruction to pull the number 25 from a spot labeled `NUM1` and the number 17 from `NUM2` into registers where the processor can work on them. The `ADD` instruction combines these two values and puts the total into register `R2`. The `ST` instruction then saves that final sum into a reserved memory space labeled `SUM`.

Program 3: Conditional Math (Add Two to Positive)

This program checks if a number is negative before performing an operation, simulating a basic "if" statement.

Code snippet

```

.ORIG x3000                ; Program starts at address x3000 [cite: 377]
LD R1, PLACE1              ; Load number from memory location PLACE1 into R1
[cite: 378, 379]
BRn DONE                  ; If the number in R1 is negative, jump to label DONE
[cite: 385]
ADD R3, R1, #2             ; Add decimal 2 to R1 and store in R3 [cite: 386, 387]
ST R3, PLACE2              ; Store the result into memory location PLACE2 [cite:
389, 390]
DONE HALT                 ; Stop execution [cite: 391]

; Data Section
PLACE2 .BLKW 1             ; Reserve one word in memory [cite: 393, 397]
PLACE1 .FILL #5            ; Initialize PLACE1 with the value 5 [cite: 394, 398]
.END                      ; End of assembly file [cite: 395, 399]

```

Explanation This logic acts as a guard for a mathematical operation. It loads a value from `PLACE1` and immediately uses a `BRn` (Branch if Negative) instruction. If the number is negative, the computer skips the math entirely and goes straight to the end. If the number is zero or positive, it proceeds to add the number 2 to it and stores that new value in `PLACE2`.

Would you like me to transcribe the 64-bit Linux assembly example from your notes as well?